TrendScope

Analyse und Vorhersage globaler Social Media Trends

Süleyman Gümüs

17. MÄRZ 2026

WebMiete24

1.Einleitung

2. Zielsetzung des Projekts

3. Analyse der Anforderungen

4. Datenbankauswahl

4.1 Mögliche Datenbanken

Für die Umsetzung des Projekts zur Analyse globaler Social Media Trends kommen verschiedene Datenbanktypen infrage. Die Auswahl hängt stark von der Art der Daten, der Skalierbarkeit sowie den Analyseanforderungen ab.

Im Folgenden werden die wichtigsten Datenbanktypen vorgestellt, die für dieses Projekt geeignet sind.

---

1. Relationale Datenbank (SQL)

Relationale Datenbanken speichern Daten in Tabellen mit festen Strukturen.

Beispiele:
- PostgreSQL
- MySQL

Eigenschaften:
- Feste Tabellenstruktur (Schema)
- Verwendung von SQL für Abfragen
- Hohe Datenkonsistenz

Einsatzmöglichkeiten im Projekt:
- Speicherung von strukturierten Daten
- Verwaltung von Trend-Rankings
- Speicherung von Analyseergebnisse

Vorteile:
- Zuverlässig und stabil
- Gut für komplexe Abfragen geeignet

Nachteile:
- Weniger flexibel bei sich ändernden Datenstrukturen
- Skalierung bei sehr großen Datenmengen aufwendiger

---

2. NoSQL-Datenbanken

NoSQL-Datenbanken sind besonders für große und unstrukturierte Datenmengen geeignet.

Beispiel:
- MongoDB

Eigenschaften:
- Flexible Datenstruktur (JSON ähnlich)
- Kein festes Schema erforderlich
- Hohe Skalierbarkeit

Einsatzmöglichkeiten im Projekt:
- Speicherung von Social Media Posts
- Verarbeitung unterschiedlicher Datenformate
- Speicherung großer Datenmengen

Vorteile:
- Sehr flexibel
- Gut für Big-Data Anwendungen geeignet

Nachteile:
- Geringere Datenkonsistenz im Vergleich zu SQL
- Komplexere Abfragen schwieriger umzusetzen

---

3. Such- und Analyse-Datenbanken

Diese Datenbanken sind speziell für die Verarbeitung und Analyse von Textdaten optimiert.

Beispiel:
- Elasticsearch

Eigenschaften:
- Volltextsuche
- Schnelle Datenanalyse
- Echtzeit Auswertung

Einsatzmöglichkeiten im Projekt:
- Erkennung von Trends und Keywords
- Analyse großer Textmengen
- Erstellung von Statistiken

Vorteile:
- Sehr hohe Geschwindigkeit bei Suchanfragen

- Ideal für Textanalyse

Nachteile:
- Aufwendigere Einrichtung
- Meist in Kombination mit anderen Datenbanken sinnvoll

---

4. Optional Graph-Datenbanken

Graph-Datenbanken werden zur Darstellung von Beziehungen zwischen Daten genutzt

Beispiel:
- Neo4j

Eigenschaften:
- Speicherung von Netzwerken und Verbindungen
- Analyse von Beziehungen

Einsatzmöglichkeiten im Projekt:
- Analyse von Zusammenhängen zwischen Trends
- Untersuchung von Hashtag Netzwerken

Vorteile:
- Ideal für Beziehungsanalysen

Nachteile:
- Für dieses Projekt nicht zwingend erforderlich
- Höhere Einarbeitungsaufwand

Zusammenfassung

Für das Projekt stehen verschiedene Datenbanktypen zur Verfügung, die jeweils unterschiedliche Stärken besitzen.

Während relationale-Datenbanken für strukturierte Daten geeignet sind, bieten NoSQL-Datenbanken Vorteile bei der Verarbeitung großer, unstrukturierter Datenmengen. Such- und Analyse-Datenbanken ermöglichen eine effiziente Auswertung von Textdaten, während Graph-Datenbanken Ideal für Beziehungsanalysen ist.

Eine Kombination dieser Systeme kann die jeweiligen Vorteile optimal nutzen.

4.2 Auswahl der geeigneten Datenbank/en

Im vorherigen Abschnitt wurden verschiedene Datenbanktypen vorgestellt.
Da das vorliegende Projekt sowohl große Mengen unstrukturierter Daten (Social Media Posts) als auch strukturierte Analyseergebnisse verarbeitet, ist der Einsatz einer einzelnen Datenbank nicht ausreichend.

Aus diesem Grund wird eine Kombination mehrerer Datenbanksysteme gewählt.

Gewählte Datenbankarchitektur

Für die Umsetzung des Projekts wird folgende Kombination verwendet:

- NoSQL-Datenbank zur Speicherung der Rohdaten
- Such und Analyse-Datenbank zur Auswertung
- Relationale Datenbank für strukturierte Ergebnisse

---

1. Verwendung einer NoSQL-Datenbank (MongoDB)

Für die Speicherung der Social Media Daten wird eine dokumentbasierte Datenbank wie MongoDB eingesetzt.

Begründung:

Social Media Daten sind:

- Unstrukturiert
- Unterschiedlich aufgebaut (je nach Plattform)
- Sehr umfangreich

MongoDB ermöglicht es, Daten im JSON-Format flexibel zu speichern, ohne ein festes Schema vorzugeben.

Vorteile im Projekt:

- Flexible Speicherung von Posts, Kommentaren und Metadaten
- Einfache Erweiterungen bei neuen Plattformen
- Gute Performance bei großen Datenmengen

---

2. Verwendung einer Such- und Analyse-Datenbank (Elasticsearch)

Für diese Analyse der gesammelten Daten wird Elasticsearch eingesetzt.

Begründung:

Das Projekt erfordert:

- Erkennung häufig verwendeter Begriffe
- Analyse von Trends in Echtzeit

- Schnelle Auswertung großer Textmengen

Elasticsearch ist speziell für solche Anforderungen entwickelt worden und bietet leistungsstarke Funktionen zur Volltextsuche und Datenaggregation.

Vorteile im Projekt:
- Schnelle Identifikation von Trends
- Echtzeit-Analyse möglich
- Optimiert für Textverarbeitung

---

3. Verwendung einer relationalen Datenbank (PostgreSQL)

Für die Speicherung der berechneten Ergebnisse wird eine relationale Datenbank wie PostgreSQL verwendet.

Begründung:

Die Ergebnisse des Systems sind klar strukturiert, bspw.:
- Top 10 Trends
- Prozentuale Verteilungen
- Trend-Scores

Solche Daten lassen sich effizient in Tabellenform speichern und auswerten.

Vorteile im Projekt:
- Strukturierte und übersichtliche Speicherung
- Einfache Abfragen (z. B. Top 10 Trends)
- Hohe Datenintegrität

Gesamtbewertung der Architektur

Die gewählte Kombination bietet folgende Vorteile:
- Trennung von Rohdaten, Analyse und Ergebnissen
- Hohe Skalierbarkeit
- Optimale Performance für jede Aufgabe
- Flexibilität für zukünftige Erweiterungen

Fazit

Für das vorliegende Projekt ist eine hybride Datenbankarchitektur die sinnvollste Lösung.

Durch die Kombination aus:
- MongoDB (Rohdaten)

- Elasticsearch (Analyse)
- PostgreSQL (Ergebnisse)

Können sowohl große Datenmengen effizient verarbeitet als auch strukturierte Ergebnisse zuverlässig gespeichert werden.

Diese Architektur entspricht modernen Standards im Bereich Big Data und stellt eine zukunftssichere Lösung dar.

5. Anforderungen an das System

Für die erfolgreiche Umsetzung des Projekts müsse verschiedene Anforderungen erfüllt werden.
Diese lassen sich in funktionale und nicht-funktionale Anforderungen unterteilen.

Funktionale Anforderungen beschreiben, was das System tun soll, während nicht-funktionale Anforderungen festlegen, wie gut das System diese Aufgaben erfüllen muss.

5.1 Funktionale Anforderungen

Die funktionalen Anforderungen definieren die Kernfunktionen des Systems.

1. Datenerfassung
   - Das System muss Daten aus Social Media Plattformen erfassen können
   - Unterstützung von APIs (z. B. TwitterX, Reddit)
   - Regelmäßige Aktualisierung der Daten

---

2. Datenspeicherung
   - Speicherung großer Datenmengen
   - Verwaltung von unstrukturierten Textdaten (Posts, Kommentare)
   - Speicherung von Metadaten (Zeit, Quelle, Interaktionen)

---

3. Datenspeicherung (NLP)
   - Analyse von Textdaten
   - Erkennung von Schlüsselwörtern und Trends
   - Sprachverarbeitung

---

4. Trendanalyse
   - Identifikation der meistgenannten Begriffe
   - Erstellung einer Top 10 Liste
   - Berechnung von Trend Scores und Prozentwerten

---

5. Prognose
    - Analyse von Entwicklungen über Zeit
    - Vorhersagen zukünftiger Trends
    - Erkennung neuer aufkommender Themen

---

6. Bereitstellung der Daten (API)
    - Bereitstellung der Ergebnisse über eine Schnittstelle
    - Kommunikation zwischen Backend und Frontend

---

7. Visualisierung
    - Darstellung der Trends im Frontend
    - Anzeige von Rankings und Diagrammen

5.2 Nicht Funktionale Anforderungen

Diese Anforderungen betreffen die Qualität und Eigenschaften des Systems.

---

1. Performance
    - Schnelle Verarbeitung großer Datenmengen
    - Kurze Ladezeiten im Frontend

---

2. Skalierbarkeit
    - System muss mit wachsenden Datenmengen umgehen können
    - Erweiterbarkeit auf weitere Plattformen

---

3. Zuverlässigkeit
    - Stabile und fehlerfreie Ausführung
    - Vermeidung von Datenverlust

---

4. Aktualität der Daten
    - Regelmäßige Updates der Trends
    - Möglichst aktuelle Ergebnisse (Near Real Time)

---

5. Sicherheit
    - Schutz der Daten vor unbefugtem Zugriff
    - Sicherer Umgang mit API-Schlüsseln

---

6. Wartbarkeit
- Saubere Code Struktur
- Erweiterbarkeit des Systems
- Einfache Fehlerbehebung

---

7. Benutzerfreundlichkeit
- Übersichtliches Frontend
- Einfache Bedienung

---

8. Datenschutz
- Einhaltung der Datenschutzbestimmungen (DSGVO)
- Keine Speicherung sensibler personenbezogener Daten

Fazit

Die definierten Anforderungen bilden die Grundlage für die Entwicklung des Systems. Sie stellen sicher, dass sowohl die funktionalen Ziele (z. B. Trendanalyse) als auch qualitative Aspekte (z. B. Performance und Sicherheit) erfüllt werden.

Durch die Berücksichtigung dieser Anforderungen kann ein leistungsfähiges, skalierbares und zukunftssicheres System entwickelt werden.

6. Betriebsmodell: Cloud vs. On-Premise

Für die Umsetzung des Projekts stellt sich die Frage, in welcher Umgebung das System betrieben werden soll.
Grundsätzlich gibt es zwei Möglichkeiten: den Betrieb in der Cloud oder den Betrieb auf eigener Hardware.

Die Wahl des Betriebsmodell hat großen Einfluss auf Skalierbarkeit, Kosten, Wartung und Flexibilität des Systems.

6.1 Cloud Lösung

Bei einer Cloud Lösung wird das System auf externen Servern eines Anbieters betrieben.

Eigenschaften:
- Nutzung von externen Rechenzentren
- Zugriff über das Internet
- Flexible Ressourcenbereitstellung

Vorteile:

- Hohe Skalierbarkeit (Ressourcen können flexibel angepasst werden)
- Geringere Wartungsaufwand (Infrastruktur wird von Anbieter verwaltet)
- Schnelle Bereitstellung von Systemen
- Gute Verfügbarkeit und Ausfallsicherheit

Nachteile:

- Anhängigkeit vom Cloud-Anbieter
- Laufende Kosten
- Daten werden extern gespeichert (Datenschutz beachten)

6.2 On-Premise Lösung

Bei einer On-Premise Lösung wird das System auf eigener Hardware betreiben, z. B. auf einem lokalen Server.

Eigenschaften:

- Betrieb im eigenen Netzwerk
- Volle Kontrolle über Hardware und Daten

Vorteile:

- Hohe Kontrolle über Daten und Systeme
- Keine Abhängigkeit von externen Anbietern
- Einhaltung von Datenschutzrichtlinien einfacher umsetzbar

Nachteile:

- Hoher Wartungsaufwand
- Kosten für Hardwarebetrieb
- Begrenzte Skalierbarkeit
- Einrichtung und Verwaltung sind komplexer

6.3 Vergleich der beiden Ansätze

Kriterium	Cloud	On-Premise
Skalierbarkeit	Sehr hoch	Begrenzt
Wartungsaufwand	Gering	Hoch
Kosten	Laufend	Anfangsinvestition
Flexibilität	Hoch	Eingeschränkt
Kontrolle	Eingeschränkt	Sehr hoch

6.4 Entscheidung für das Projekt

Für das vorliegende Projekt wird eine Cloud-basierte Lösung bevorzugt.

Begründung:

Das System verarbeitet große Mengen an Social Media Daten, die kontinuierlich wachsen.
Eine Cloud Lösung bietet die notwendige Skalierbarkeit, um diese Daten effizient zu verarbeiten.

Zudem ermöglicht die Cloud:

- Eine schnelle Entwicklung und Bereitstellung
- Einfache Integration von Datenbanken und Analyse-Tools
- Flexible Anpassung der Ressourcen je nach Bedarf

Fazit

Die Analyse zeigt, dass sowohl Cloud als auch On-Premise Lösungen ihre jeweiligen Vor- und Nachteile besitzen.

Für datenintensive Anwendungen wie die Analyse von Social Media Trends ist jedoch die Cloud aufgrund ihrer Skalierbarkeit und Flexibilität die geeignetere Wahl.

Die Entscheidung für eine Cloud Lösung bildet somit eine wichtige Grundlage für die weitere Systemarchitektur und Umsetzung des Projekts.

7. Systemarchitektur

Die Systemarchitektur beschreibt den grundlegenden Aufbau des Systems sowie das Zusammenspiel der einzelnen Komponenten.
Ziel ist es, eine strukturierte und skalierbare Lösung zur Analyse von Social Media Daten zu entwickeln.

Das System folgt einem modularen Aufbau, bei dem jede Komponente eine klar definierte Aufgabe übernimmt.

7.1 Architekturdiagramm

Social Media APIs
↓
Data Collector (Python)
↓
MongoDB (Rohdaten)
↓
NLP & Analyse (Python)
↓
Elasticsearch (Analyse)
↓

PostgreSQL (Ergebnisse)
↓
FastAPI Backend
↓
Frontend (HTML/CSS/JS)

8. Umsetzung / Realisierung

In diesem Abschnitt wird die konkrete Umsetzung des Projekts beschrieben.
Ziel die Entwicklung eines Systems zur Analyse und Vorhersage globaler Social Media Trends unter Verwendung moderner Web- und Datenverarbeitungstechnologien.

Die Umsetzung erfolgt schrittweise und orientiert sich an der zuvor definierten Systemarchitektur.

8.1 Verwendete Technologien

Für die Realisierung des Projekts werden folgende Technologien eingesetzt:

Backend:
- Programmiersprache: Python
- Framework: FastAPI
- Datenverarbeitung: spaCy / NLTK
- Machine Learning: scikit-learn

Datenbanken:
- MongoDB (Rohdaten)
- Elasticsearch (Analyse)
- PostgreSQL (Ergebnisse)

Frontend:
- HTML
- CSS
- JavaScript

8.2 Datenbeschaffung (Data Collection)

Die Datenerfassung erfolgt über Schnittstellen (APIs) von Social-Media-Plattformen.

Vorgehen:
- Verbindung zu APIs herstellen (z. B. Twitter, Reddit)
- Abrufen von Posts und Kommentaren
- Speicherung der Daten in der Datenbank

Ziel:

Kontinuierliche Sammlung aktueller Daten zur Trendanalyse.

8.3 Datenspeicherung

Die gesammelten Daten werden strukturiert gespeichert:

- Rohdaten → MongoDB
- Analyse-Daten → Elasticsearch
- Ergebnisse → PostgreSQL

Beispiel gespeicherter Daten:

- Textinhalt eines Posts
- Zeitstempel
- Plattform (Quelle)
- Interaktionen (Likes, Kommentare)

8.4 Datenverarbeitung (NLP)

Die Textdaten werden mit Natural Language Processing (NLP) analysiert.

Schritte:

- Entfernen von Stopwörtern
- Tokenisierung (Zerlegung in Wörter)
- Erkennung relevanter Keywords
- Vereinheitlichung von Begriffen

Ziel:
Extraktion von bedeutungsvollen Informationen aus unstrukturierten Texten.

8.5 Trendanalyse und Scoring

Die erkannten Keywords werden ausgewertet, um Trends zu identifizieren.

Vorgehen:

- Zählen der Häufigkeit von Begriffen
- Analyse der zeitlichen Entwicklung
- Berechnung eines Trend-Scores

Ergebnis:

- Top 10 Trends
- Prozentuale Verteilung
- Ranking der Themen

8.6 Prognose von Trends

Zur Vorhersage zukünftiger Trends werden einfache Machine Learning Methoden eingesetzt

Methoden:

- Analyse historischer Daten
- Berechnung von Wachstumsraten
- Zeitbasierte Vorhersagen

Ziel:
Erkennung von Trends, die in Zukunft an Bedeutung gewinnen könnten.

8.7 Backend-Entwicklung

Das Backend wird mit FastAPI umgesetzt.

Funktionen:

- Bereitstellung von API-Endpunkten
- Kommunikation mit Datenbanken
- Verarbeitung von Anfragen vom Frontend

Beispiel:

- /trends → liefert aktuelle Top 10
- /forecast → liefert Prognosen

8.8 Frontend-Entwicklung

Das Frontend dient zur Visualisierung der Ergebnisse.

Funktionen:

- Anzeige der Top 10 Trends
- Darstellung von Diagrammen
- Übersichtliche Benutzeroberfläche

Technologien:

- HTML für Struktur
- CSS für Design
- JavaScript für Interaktivität

8.9 Ablauf des Gesamtsystems

1. Daten werden über APIs gesammelt
2. Speicherung in der Datenbank
3. Verarbeitung durch NLP

4. Analyse und Bewertung der Trends
5. Bereitstellung über API
6. Darstellung im Frontend

Fazit

Die Umsetzung zeigt, wie verschiedene Technologien kombiniert werden können, um ein komplexes System zur Analyse von Social-Media-Daten zu realisieren.
Durch die modulare Struktur ist das System erweiterbar und kann in Zukunft um weitere Funktionen ergänzt werden.

---

9. Prognose (ML)

10. Testergebnisse

11. Fazit